

Aurum Market: búsqueda semántica y control de catálogo

Manuel Pérez

Evaluación · Bases de Datos Vectoriales

1 de septiembre de 2026

Resumen — El sistema entregado convierte los 15.000 productos de Aurum Market en un servicio de descubrimiento medible: recupera por intención, filtra por marca dentro de la propia consulta a la base de datos, absorbe altas, actualizaciones y bajas sin duplicar registros, y detecta altas potencialmente duplicadas con una regla calibrada solo con datos de desarrollo. El motor elegido es **Qdrant** (local, Docker) con su SDK nativo; la representación ganadora es **multilingual-e5-large** codificando **título + marca + color** en lugar del campo `text` completo — decisiones que salieron de una rejilla de experimentos (composición, objetivo de entrenamiento y capacidad, con BM25 como baseline, dos retadores externos y Gemini Embedding 2 preparado como opción API) y de una **validación ampliada** sobre 413 consultas públicas de ESCI con estadística pareada, que invirtió el veredicto de las 8 consultas de desarrollo. Cada cifra de este informe se regenera desde `config/run_config.yaml` y queda justificada, con tablas y gráficos, en la serie de notebooks de I+D `actividad_00-05`.

1 Contrato del sistema y baseline

El sistema expone una interfaz única (`DiscoveryService` y la CLI `make search`) que recibe una consulta y devuelve resultados normalizados: `product_id`, posición, título, marca, `score` nativo y su semántica (`similarity`, mayor es mejor). Los dos recorridos de negocio comparten esa interfaz:

- **Descubrimiento:** top- k configurable, con filtro opcional de marca que viaja dentro de la consulta a la base de datos.
- **Control de altas:** la ficha entrante se codifica como documento, la base vectorial genera los candidatos y una regla de umbral decide si existe duplicado, señalando el `product_id` concreto.

Como referencia interpretable se implementó un **baseline léxico BM25** [6] ($k_1=1,5$, $b=0,75$, tokenización en minúsculas y sin tildes) sobre el mismo corpus. El baseline no es decorativo: fija el listón que la representación densa debe superar y, como se ve en la sección 2, no lo supera en todas las métricas, lo que delimita honestamente dónde aporta la semántica.

La figura 1 resume la arquitectura: datos → representación (embeddings con manifiesto SHA-256) → Qdrant (colección persistente con HNSW y payload indexado) → servicios de búsqueda y duplicados → evaluación y artefactos.

2 Representación vectorial

Texto codificado. Cada producto se representa con una composición documentada y saneada: los valores vacíos son información ausente (nunca la cadena "nan") y los campos vacíos se omiten. Se compararon dos composiciones y seis modelos locales [2, 3, 4], más el baseline léxico, siempre con búsqueda exacta para aislar la representación del índice (tabla 1); una configuración adicional con Gemini Embedding 2 (API de la sesión 01) queda preparada y se construye solo si existe `GEMINI_API_KEY`. El enunciado no obliga a modelos locales ("no existe ventaja por utilizar cloud

frente a local"); el final local es una decisión de diseño: evaluación sin coste para el corrector y regeneración completa sin clave ni binarios grandes en el repositorio.

Tabla 1: Experimentos de representación sobre las 8 consultas de desarrollo (búsqueda exacta). En negrita, la configuración final — que **no** es la ganadora de esta tabla: la eligió la validación ampliada (tabla 2).

Experimento	nDCG@10	Recall@10	MRR@10
bm25_full_text	0.558	0.187	0.646
e5_small_full	0.533	0.205	0.588
e5_small_title	0.555	0.223	0.698
e5_base_title	0.563	0.248	0.792
e5_large_title	0.527	0.195	0.719
bge_m3_title	0.637	0.266	0.875
qwen3_embedding_title	0.648	0.296	0.750
minilm_full	0.273	0.095	0.310

Dos conclusiones salen ya de la tabla de desarrollo:

1. **El campo `text` completo perjudica a E5.** Las fichas arrastran *keyword-stuffing* (títulos repetidos, listas de variantes, HTML residual). Codificar solo título+marca+color mejora las tres métricas frente al texto completo (+0,02 nDCG, +0,018 recall, +0,11 MRR). La suciedad no se corrigió a mano: se neutralizó eligiendo qué codificar, que es una decisión trazable del experimento.
2. **Cambiar de modelo se analizó, no solo se citó.** MiniLM, sin prefijos y entrenado para paráfrasis genérica, se hunde (nDCG 0.273): confirma que el prefijo `query:/passage:` y el entrenamiento de recuperación de E5 son la parte que importa, no la dimensión (ambos son 384d).

La tercera conclusión exigió más datos. Sobre 8 consultas cada una mueve la macro-media 0,125: la tabla sugiere que e5-large (el mayor de la familia) *empeora* a e5-base y que los retadores bge-m3 y Qwen3-Embedding dominan.

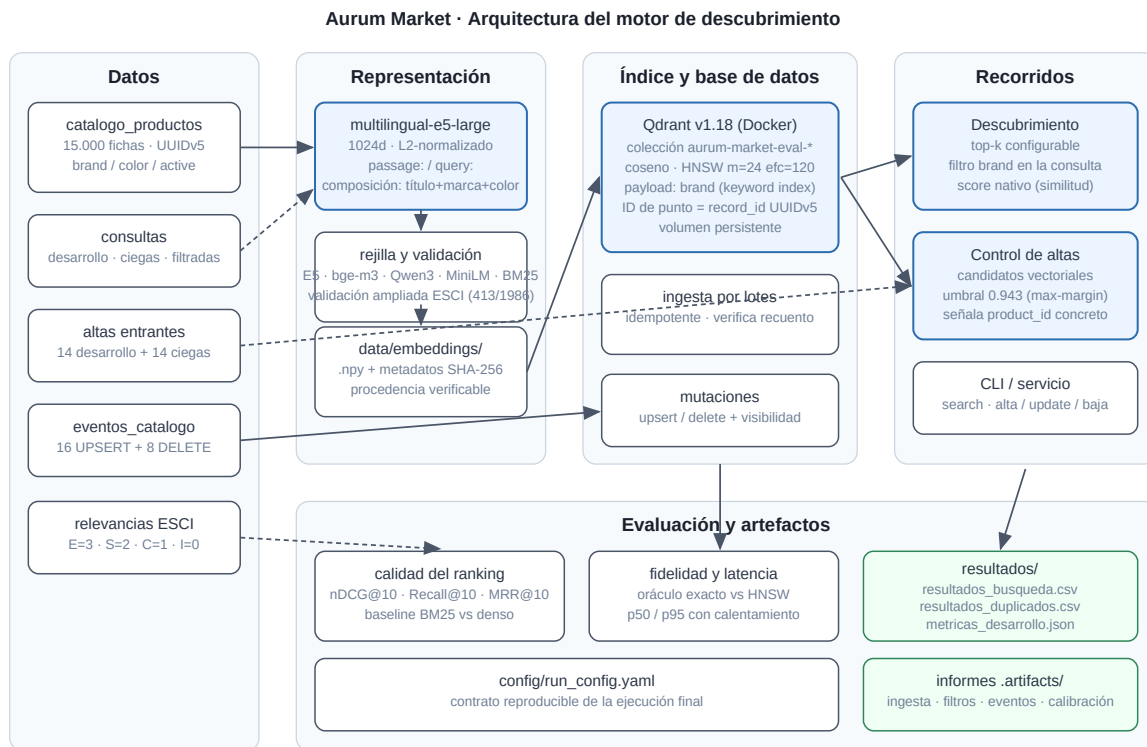


Figura 1: Arquitectura del motor de descubrimiento y control de catálogo.

Antes de fijar la configuración se construyó una **validación ampliada** (`make validate-challengers`): el snapshot de la actividad deriva del dataset público ESCI [1], así que pueden tomarse todas las consultas españolas publicadas —excluidas las 8 de desarrollo y las 4 consultas base de las 12 de evaluación ciega—, restringir sus juicios al catálogo de 15.000 y evaluar con el mismo oráculo exacto: 413 consultas con ≥ 5 productos juzgados en catálogo (nivel de decisión) y 1.986 con ≥ 3 (robustez). La comparación contra e5-base es **pareada**: delta medio por consulta, IC bootstrap al 95 % y p-valor de permutación por signos (tabla 2).

Tabla 2: Validación ampliada (413 consultas, oráculo exacto). Δ = delta medio pareado frente a e5_base_title (permutación por signos, 10.000 remuestreos, semilla 42); * = $p < 0,05$ nominal, ** = sobrevive además la corrección de Holm sobre los 18 contrastes del nivel.

Configuración	nDCG@10	Recall@10	MRR@10
e5_large_title	0.469**	0.473**	0.558*
e5_small_title	0.451*	0.449	0.549
qwen3_embedding	0.450	0.462**	0.533
bge_m3_title	0.447	0.449	0.540
e5_base_title	0.432	0.432	0.530

El conjunto grande invierte el veredicto del pequeño, dos veces. **e5-large, el “peor” en desarrollo, es el único modelo cuya mejora sobre e5-base sobrevive la corrección de Holm en nDCG y recall en el nivel de decisión** (p corregido 0,002; su MRR queda en evidencia solo nominal, $p=0,040$). Con $n=1986$ el veredicto es total: las tres métricas sobreviven Holm (0,002) y e5-large gana los **pareados**

directos —persistidos en el propio artefacto, no calculados a mano— contra bge-m3 y Qwen3 también en las tres (p corregido $\leq 0,008$); con 413 consultas esos directos apuntan igual pero sin fuerza concluyente. Y el dominio aparente de los retadores era en buena parte ruido de muestra (bge-m3 frente a e5-base con $n=413$: $p=0,13/0,13/0,51$). Los juicios restringidos al catálogo son dispersos y deprimen los valores absolutos por igual para todos los modelos; por eso la decisión se apoya en las comparaciones pareadas, no en los niveles. La lección metodológica queda a la vista en la tabla 1: la configuración final *pierde* sobre las 8 consultas de desarrollo, y se acepta ese número porque la evidencia con 50× más consultas demuestra que es la muestra la que engaña — sin que eso restaure una escalera de capacidad limpia: e5-small-title también supera a e5-base en el conjunto grande.

Métrica, normalización y score. Los embeddings se L2-normalizan al codificar y la colección usa distancia **coseno**; con vectores unitarios el score devuelto por Qdrant es la similitud coseno en $[-1, 1]$ y “mayor es mejor”. Esa semántica se conserva de extremo a extremo (`score_kind="similarity"` en el contrato de resultados) y nunca se compara con distancias.

Los embeddings se persisten en `data/embeddings/` con un manifiesto que fija modelo, prefijos, composición, dimensión y checksums SHA-256 de entradas y salidas, de modo que cualquier matriz puede auditarse contra el CSV del que salió.

3 Índice y base de datos

Elección del motor. El enunciado no impone proveedor y excluye el ranking cronometrado entre motores, así que la elección es por criterios. De los cinco propuestos, Qdrant es el único que combina parámetros HNSW explícitos por colección y *por consulta*, un **oráculo exacto sobre la misma colección** (`exact=true`) —la base de toda la medición de fidelidad ANN de este trabajo—, observabilidad del estado del índice (`indexed_vectors_count`, sin la cual la lección de producción de más abajo habría sido invisible), filtrado dentro del plan de búsqueda por índice de payload, IDs de punto UUID que casan con el contrato UUIDv5 del manifiesto, y despliegue local en un contenedor sin credenciales ni costes para quien corrige. Pinecone oculta la decisión de índice; Milvus permitiría comparar familias (IVF, PQ) pero con una huella operativa injustificable para 15.000 productos. Elegido el motor, **la familia ANN viene dada —Qdrant solo implementa HNSW—** y la restricción no duele: IVF exigiría re-entrenar centroides ante las altas incrementales del control de catálogo (§5), y PQ distorsionaría el score coseno del que depende el umbral de duplicados (§6); el notebook 02 desarrolla el argumento con la tabla de criterios completa.

Esquema. Una colección Qdrant [8] (`aurum-market-eval-catalogo`) con un vector sin nombre de 1024d, distancia coseno, e ID de punto igual al `record_id` UUIDv5 del catálogo —el mismo UUID que el manifiesto de datos declara, verificado en carga—. El payload conserva `product_id`, `title`, `brand`, `color`, `locale`, `catalog_version` y `active`; `brand` tiene índice de payload `keyword` creado junto con la colección para que el filtrado sea parte del plan de búsqueda. Los valores ausentes se guardan como cadena vacía, siempre con el mismo criterio.

Configuración ANN explícita. HNSW [5] con `m=24`, `ef_construct=120` y `ef_search=128`, fijados en `config/run_config.yaml`.

Una lección de producción: el índice que no existía. La primera versión funcional del sistema pasaba todas las pruebas y reportaba fidelidad ANN 1.0... porque no había índice. Qdrant solo construye el grafo HNSW cuando un segmento supera `indexing_threshold` (10–20 MB por defecto), y el catálogo de entonces (15,000 × 384 float32) repartido en 4 segmentos no lo alcanzaba: estado *green* con `indexed_vectors_count=0`, y cada búsqueda “ANN” era en realidad un escaneo exhaustivo con `ef_search` inerte. Una revisión adversarial del código lo detectó verificándolo contra la colección viva. La corrección tiene dos partes: `indexing_threshold=100 KB` al crear la colección, y una verificación tras la ingesta —previa a servir consultas del pipeline— que exige estado *green* y una cobertura mínima del 80 % de vectores indexados (esperar solo el estado verde no detecta nada, porque verde no implica indexado; en la ejecución final la cobertura es 15.000/15.000). Tras la corrección, las latencias del escaneo se redujeron aproximadamente a la mitad con la configuración de entonces (384d: p50 de ~4,3 a ~2,3 ms). La moraleja queda en el sistema: la verificación del estado de indexación debe comprobar el índice, no el semáforo.

Ingesta por lotes e idempotente. Lotes de 256 con `wait=True` y `upsert` por `record_id`: repetir la ingesta completa deja exactamente 15.000 registros (verificado en el informe de ingesta: `count_before=count_after=15000`, `idempotent=true`). La ingesta completa tarda ~8 s en local y la colección puede reconstruirse desde cero con `AURUM_ALLOW_RESET=true`.

4 Recuperación y filtros

La búsqueda global devuelve *top-k* con score nativo; la filtrada añade una condición `brand equals <valor>` dentro de `query_points` (filtrado del lado del servidor, sobre el índice de payload), nunca un post-filtrado del *top-10* global. Las cuatro consultas filtradas devuelven 10/10 resultados que cumplen la marca (Einhell, Apple, NIKE, SAM-SUNG), verificado automáticamente en el informe de filtros de resultados/evidencia/.

Los casos límite tienen tratamiento explícito y probado: una colección vacía lanza un error accionable (“ingiere con `make ingest`”), un filtro sin resultados devuelve lista vacía sin error (sondeado con una marca inexistente: 0 resultados), y un Qdrant caído se traduce en `VectorStoreUnavailableError` con instrucciones, en todas las operaciones (búsqueda, ingesta, lecturas, borrados).

5 Operaciones sobre el catálogo

Los 24 eventos de `eventos_catalogo.csv` (8 actualizaciones, 8 altas, 8 bajas) se aplican en orden de *sequence* sobre una colección dedicada sembrada con el catálogo íntegro, de modo que los artefactos entregados —generados sobre el catálogo prístino— siguen siendo reproducibles. Ambas colecciones comparten esquema, configuración e ingesta.

- **Idempotencia:** la secuencia completa se aplica **dos veces**; el recuento final es 15.000 en ambas pasadas (8 altas – 8 bajas) y el estado final se verifica *registro a registro*: los 24 `record_id` afectados se leen por ID y muestran la versión entregada o han dejado de existir.
- **Visibilidad:** para una operación de cada tipo se verifica lectura por ID y consulta vectorial con espera activa acotada (`wait_until` con *timeout*). Con `wait=True` en las escrituras, la actualización muestra `catalog_version=2` y aparece en el *top-3* de su propio vector al primer intento (~3 ms); el alta es recuperable por ambas rutas; la baja deja de serlo. El sistema no cronometra proveedores: demuestra que una escritura confirmada acaba siendo observable y que sabe esperar o fallar con *deadline* si no lo es.

Operaciones en vivo. Los tres tipos de operación pueden además provocarse interactivamente (`make alta/update/baja`) sobre una colección *sandbox* sembrada del catálogo prístino. El alta pasa primero por la regla de duplicados de la sección 6 y queda bloqueada para revisión humana si la regla la marca; la regla vigila solo fichas nuevas, porque el catálogo contiene variantes casi gemelas legítimas y editar una identidad existente no debe bloquearse por un gemelo preexistente.

Seguridad de las operaciones. Toda operación válida

que la colección empiece por el prefijo `aurum-market-eval` antes de construir el cliente; recrear una colección exige `AURUM_ALLOW_RESET=true`; borrarla exige la confirmación exacta `AURUM_CONFIRM_CLEANUP=DELETE:<colección>`. Todo desactivado por defecto, sin credenciales en el repositorio y sin servicios cloud: el corrector no hereda costes.

6 Control de altas duplicadas

Regla. La base vectorial genera los 5 candidatos más próximos del alta entrante (codificada como documento, con la misma composición que el catálogo). Se declara duplicado si el mejor candidato alcanza $score \geq 0,943$; el margen sobre el segundo candidato se registra como evidencia (umbral de margen 0,0: la rejilla lo descarta, porque hay duplicados reales casi empatados con una segunda copia legítima).

Calibración solo con desarrollo. Sobre los 14 casos etiquetados, la rejilla explora todos los umbrales que separan scores observados. Los 7 duplicados puntúan en $[0,988, 1,000]$ y los 7 no duplicados en $[0,874, 0,898]$ — con `e5-large` el hueco es más ancho que con `e5-base` (0.090 frente a 0.075): el modelo promovido también separa mejor los duplicados—. Cualquier umbral de score en ese hueco logra $F_1=1,0$. Se eligió un umbral **centrado en el margen de separación** (0.943, punto medio 0.9431) en lugar del umbral máximo que devuelve la rejilla (0.9883), criterio *max-margin* que no se pega a ningún caso concreto y generaliza mejor. Con él: precisión 1.0, recall 1.0, F_1 1.0 en desarrollo, exigiendo además que el candidato señalado sea exactamente el producto de referencia etiquetado.

Costes asimétricos. Un falso positivo retiene una ficha legítima en revisión: molesta al vendedor y añade trabajo manual, pero es reversible en minutos. Un falso negativo publica un duplicado que fragmenta reseñas y stock y degrada el propio buscador; su coste es mayor y más silencioso. Por eso el desempate de la calibración prima la precisión (la cola de revisión debe ser creíble) pero el umbral elegido queda centrado, sin sacrificar recall. En desarrollo no hubo ni un tipo de error ni el otro; el caso más difícil (DEV-NEW-007, score 0.898, un no-duplicado muy parecido) queda a 0.045 del umbral y es el que habría que vigilar en producción.

Evaluación ciega. Sobre `altas_evaluacion.csv` la regla predice 6 duplicados y 8 altas nuevas. Trece de los catorce casos replican la separación de desarrollo (positivos en $[0,980, 1,000]$, negativos en $[0,866, 0,889]$); el decimo-cuarto (EVAL-DUP-004) puntúa **0.9428, dentro del hueco de desarrollo y a 0.0002 del umbral**, y la regla lo clasifica negativo. Las etiquetas del ciego están reservadas, pero la convención de identificadores (EVAL-DUP-* frente a EVAL-NEW-*) indica que se trata casi con certeza de un duplicado real: un probable **falso negativo**, el error caro. La lectura honesta del ciego es precisión estimada 6/6 y recall estimado 6/7 ($\sim 0,86$), no una separación perfecta. El umbral no se toca aun así: el conjunto de calibración no contiene ningún ejemplo en esa zona y moverlo *a posteriori* sería calibrar con el conjunto ciego — exactamente lo que la metodología declaró que no haría. Ese score intermedio es la definición operativa de “revisión humana”: la mejora natural de la regla no es otro umbral, sino una banda de abstención alrededor de la frontera (véase la sección 9).

7 Evaluación

Todas las métricas se regeneran con un único comando (`make metrics`) y quedan en `resultados/metricas_desarrollo.json`; cada experimento conserva configuración, métricas e IDs recuperados.

Declaración de relevancia (fijada antes de los experimentos y sin cambios): $nDCG@10$ [7] usa ganancias $E=3$, $S=2$, $C=1$, $I=0$; **Recall@10 considera relevantes EUS** con el conjunto relevante completo como denominador (con hasta 39 relevantes por consulta, el techo alcanzable con $k=10$ es $< 1,0$; se reporta sin maquillar el denominador); **MRR@10 considera relevante solo E**. La tabla 3 resume la ejecución final.

Tabla 3: Métricas de la configuración final medidas a través de Qdrant (ruta de producción). Latencias: 12 consultas $\times$ 30 repeticiones tras 5 de calentamiento, en local (WSL2, 8 vCPU); varían $\pm 0,3$ ms entre ejecuciones y no comparan proveedores.

Métrica	Valor
$nDCG@10$	0.527
Recall@10 (EUS)	0.195
MRR@10 (E)	0.719
Latencia p50 / p95	2.72 / 3.05 ms
Fidelidad ANN@10 (media / mín.)	1.000 / 1.000

La fidelidad es una medición, no una tautología. Tras el incidente de la sección 3, la fidelidad —fracción del top-10 exacto que recupera el ANN— se valida con un barrido de `ef_search` (`make sweep-ef`) sobre dos grafos del mismo catálogo: el entregado ($m=24$, optimizado) y uno deliberadamente débil ($m=4$, `ef_construct=16`) donde el compromiso emerge (tabla 4).

Tabla 4: Barrido de `ef_search`: fidelidad frente al oráculo exacto (20 consultas).

Grafo	ef	Fid. media@10	Fid. mín.@10
$m=24$ (entregado)	10	0.905	0.30
$m=24$ (entregado)	64	0.995	0.90
$m=24$ (entregado)	128	1.000	1.000
$m=4$ (débil)	10	0.490	0.00
$m=4$ (débil)	128	0.890	0.40

Dos lecciones: en el grafo entregado, con 1024 dimensiones y el barrido medido justo la ingesta, `ef_search` es un dial real —`ef=10` pierde de media un 10 % del top-10 (mínima 0.30 en una consulta) y solo `ef=128` recupera fidelidad 1.0—, y un grafo mal construido **no se arregla buscando más**: con $m=4$ hay una consulta que pierde el top-10 *entero* a `ef=10` y la fidelidad media se estanca en $\sim 0,89$ incluso a `ef=128`, porque las aristas que no existen no se pueden recorrer. La calidad se decide en construcción (m , `ef_construct`); al crecer el catálogo este barrido es la herramienta de re-calibración.

8 Atribución de errores

Tres fallos representativos, con la capa responsable determinada por evidencia. La fidelidad ANN 1.000 en la configuración final descarta el índice en todos ellos: el vecino

que devuelve HNSW es el vecino exacto (verificado consulta a consulta en el notebook 05).

Caso 1 — Representación: «estantes sin taladro habitación» (38249, nDCG 0.246). Tres libros encabezan el ranking (*La buena cocina sin sal*, *Ángeles sin alas*, *Fragile Rooms*) y el primer estante E no llega hasta la 4ª posición (MRR 0.25). El fallo recorre toda la escalera de capacidad —e5-small también puntuaba MRR 0.25, e5-base limpiaba la cabeza pero no la cola, y e5-large vuelve a tropezar—: E5 ancla la negación “sin + sustantivo” y con títulos cortos sin marca el parecido superficial domina; más parámetros no lo desprenden. El oráculo exacto devuelve lo mismo: el vecino exacto ya es semánticamente malo, luego el error nace en la representación, no en el índice. Mitigación razonable: componer el texto con una frase de categoría o un reranking léxico ligero del top-50.

Caso 2 — Datos y juicios: «disfraz halloween talla grande hombre» (33633, MRR 0.0). El sistema llena el top-10 de disfraces de Halloween reales y aun así puntúa nDCG 0.042 con recall y MRR 0: la consulta tiene 16 juicios (solo 4 relevantes EUS) y el único Exact etiquetado es... una blusa de mujer (ruido del dataset ESCI [1]): el MRR es 0 precisamente porque el sistema *no* recupera esa blusa, y los disfraces plausibles sin juicio computan ganancia 0. Es un error de la capa de datos/etiquetas que ninguna configuración puede remontar: en toda la rejilla, ninguna logra aquí MRR > 0.

Caso 3 — Representación en atributos finos: «botines marrones mujer tacon medio» (18868, nDCG 0.474). La cabeza es buena (dos E marrones, MRR 1.0), pero en el top-5 conviven unas botas *negras de caña alta* y *tacón alto* etiquetadas Irrelevant y unos zapatos de tacón alto sin juicio: la categoría se acierta, pero color y altura de tacón se difuminan en la similitud coseno — y el patrón persiste en los tres tamaños de la familia E5, señal de que no es cuestión de capacidad. La mitigación estructural no es cambiar de modelo sino mover atributos duros (color) a metadatos filtrables, como ya se hace con la marca.

Las otras dos capas, con su propia evidencia. La capa de **índice** se demuestra provocándole una pérdida real: en el grafo débil del barrido ($m=4$, $ef_search=10$) el ANN llega a perder el top-10 completo que el oráculo recupera en una consulta, exactamente la forma de evidencia que define el enunciado; en la configuración entregada la pérdida es cero. En **persistencia/consistencia** no se observó discrepancia alguna: las escrituras usan `wait=True`, las tres verificaciones de visibilidad de la sección 5 convergen al primer intento y el estado final de los 24 eventos se valida registro a registro.

9 Decisión recomendada y evolución

Para Aurum Market hoy: Qdrant local con HNSW ($m=24$, $ef_construct=120$, $ef_search=128$), multilingual-e5-large sobre título+marca+color, filtro de marca por payload indexado, regla de duplicados con umbral 0.943 sobre score coseno y revisión humana de los positivos. El coste de e5-large no lo paga el índice (mismos ~2,7 ms de p50 en Qdrant) sino el encoder de cada consulta interactiva, medido en el notebook 02 y asumible para una

CLI. Todo el recorrido corre en un portátil, es idempotente y se reconstruye desde cero con dos comandos.

Qué cambiaría al crecer el catálogo (15k → 1M):

- **Índice:** el barrido de `ef_search` pasa de curiosidad a herramienta de operación; habría que re-medir fidelidad por percentil de consulta y presupuestar memoria del grafo (o evaluar cuantización escalar de Qdrant).
- **Representación:** el caso 1 de la sección 8 escala mal y sobrevive a toda la familia E5; el siguiente paso no es otro encoder sino un reranker sobre el top-100 o señal de categoría en el texto, re-evaluados con el arnés de la validación ampliada (que ya conserva IDs, configuraciones y estadística pareada). La opción API (Gemini Embedding 2) queda preparada.
- **Duplicados:** con más altas diarias, el umbral único debería convertirse en dos (auto-rechazo / revisión humana / auto-aprobación) calibrados sobre la curva precision-recall completa — el caso ciego a 0.0002 de la frontera es el argumento empírico de esa banda de abstención.
- **Operativa:** la colección de eventos separada se sustituiría por *snapshots* de Qdrant y un flujo de reingesta azul/verde; los interruptores de limpieza ya existen y se mantendrían.

El objetivo no era demostrar que una base vectorial devuelve algo: era saber cuándo sus resultados son útiles (consultas con intención clara y atributos blandos), qué capa explica sus errores (representación y juicios, no el índice) y qué decisión resiste producción (la configuración completa está en `config/run_config.yaml` y cada número de este informe se regenera desde ella).

A Reproducción y artefactos

Reproducción íntegra (entorno limpio; los embeddings de la rejilla completa dominan el tiempo — minutos con GPU): `make setup && make up && make embeddings && make pipeline`. La validación ampliada se regenera con `make validate-challengers` tras una única descarga del parquet público de ESCI (51 MB; la URL la imprime el propio script).

Notebooks de I+D (entregados ejecutados). La serie `notebooks/actividad_00–05` justifica cada bloque de decisiones contra el sistema vivo —caso/datos/baseline, representación, índice y BD, recuperación y filtros, operaciones y duplicados, y evaluación completa con atribución de errores y el checklist del enunciado verificado por aserciones— y explica cada concepto en su primera aparición para lectura secuencial. Se regeneran con `make notebook` y se ejecutan *headless* con `make execute-notebook`.

Artefactos entregados.

`resultados/resultados_busqueda.csv`

Top-10 por consulta ciega (12 × 10 filas, IDs únicos por consulta).

`resultados/resultados_duplicados.csv`

Decisión, candidato y score de las 14 altas ciegas.

`resultados/metricas_desarrollo.json`

nDCG/Recall/MRR@10, latencias p50/p95, fidelidad ANN, duplicados y entorno.

config/run_config.yaml

Configuración exacta de la ejecución final.

docs/images/arquitectura.svg

Diagrama de arquitectura (figura 1).

resultados/evidencia/

Copia versionada de la evidencia: registro de experimentos con IDs recuperados, validación ampliada con estadística pareada, evaluación completa con entorno, barriado ef_search, calibración y decisiones de duplicados, informes de filtros, eventos e ingesta.

.artifacts/

Los mismos informes en su ubicación de trabajo (regenerables con make pipeline).

Verificación previa a la entrega. Ingesta repetida sin aumentar recuento ✓ · filtros nunca devuelven otra marca ✓ · eventos dejan el estado esperado dos veces ✓ · 12 rankings ciegos con 10 IDs únicos y válidos ✓ · toda predicción positiva señala candidato ✓ · métricas regenerables con make metrics ✓ · sin claves ni datos reservados en el repositorio ✓ (55 pruebas automatizadas: make test, make test-integration).

Referencias

- [1] C. K. Reddy *et al.* (2022). Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search.

arXiv:2206.06588.

- [2] L. Wang *et al.* (2024). Multilingual E5 Text Embeddings: A Technical Report. *arXiv:2402.05672.*
- [3] N. Reimers y I. Gurevych (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv:1908.10084.*
- [4] J. Chen *et al.* (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings. *arXiv:2402.03216.*
- [5] Y. A. Malkov y D. A. Yashunin (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 42(4). *arXiv:1603.09320.*
- [6] S. Robertson y H. Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4).
- [7] K. Järvelin y J. Kekäläinen (2002). Cumulated gain-based evaluation of IR techniques. *ACM TOIS*, 20(4).
- [8] Qdrant. Documentación: colecciones, HNSW y filtrado por payload. <https://qdrant.tech/documentation/concepts/>.